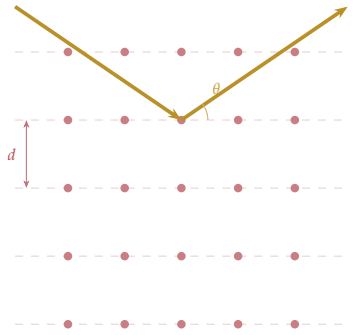
CONSTRUCTOR MATHEMATICAL JOURNAL

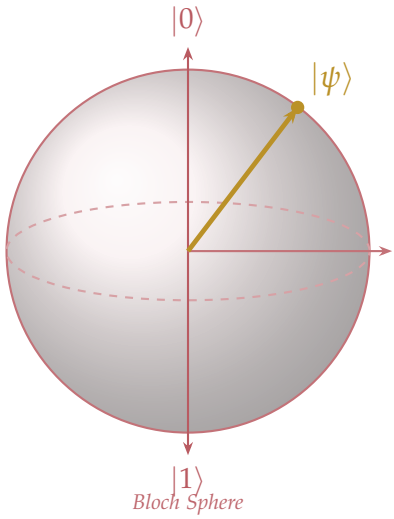
VOLUME 1 • SPRING 2025

Constructor University Mathematics Society

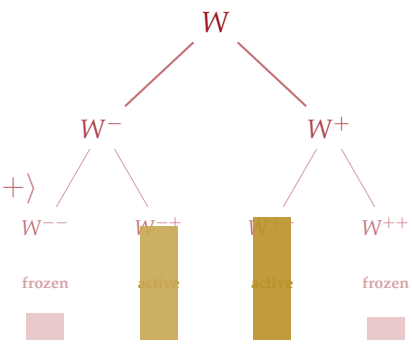


$2d \sin\theta = n\lambda$

Bragg's Law



Bloch Sphere



Channel Polarization

IN THIS ISSUE

Shannon's Theorem and Polar Codes
Aly Assaf

The Qubit and Shor's Algorithm
Ahmed Maghri

Bragg's Law and X-Ray Crystallography

ABOUT THE COVER

Left: Bragg's Law ($2d \sin\theta = n\lambda$) describes X-ray diffraction from parallel crystal planes, the technique at the heart of X-ray crystallography and protein structure determination. *Centre:* The Bloch sphere parametrises the state space of a qubit $|\psi\rangle$, the fundamental building block of Shor's quantum factoring algorithm.

Constructor University Mathematics Society • Bremen, Germany

Inaugural Edition • Spring 2025 • Volume 1

XXX -- math.constructor.university

Letter from the Editors

Dear reader,

It is with great pleasure that we present the inaugural edition of the *Constructor Mathematical Journal*, the student journal of the Constructor University Mathematics Society.

This journal was born from a desire to give undergraduate researchers a venue to communicate their work and to foster a culture of mathematical inquiry at Constructor University Bremen. Each article collected here originated in our Mathematics Undergraduate Seminar and has been developed into an expository piece intended to be accessible to a wide audience.

In this first volume you will find three articles spanning information theory, coding theory, quantum computation, and mathematical physics:

- **Shannon's Theorem and Polar Codes** explores the foundations of reliable communication laid by Claude Shannon and traces the path to Erdal Arıkan's breakthrough construction of polar codes.
- **The Qubit and Shor's Algorithm** provides a step-by-step journey from the basics of quantum computing to Peter Shor's celebrated factoring algorithm and its implications for modern cryptography.
- **Bragg's Law and X-Ray Crystallography** *(description placeholder — to be completed once the third article is finalised).*

We hope that these articles inspire curiosity and further study. We thank all authors, reviewers, and the Constructor University faculty for their support.

The Editorial Board

XXX — Editor-in-Chief	Constructor University Bremen
XXX — Managing Editor	Constructor University Bremen
XXX — Associate Editor	Constructor University Bremen

Constructor Mathematical Journal is published by the Constructor University Mathematics Society, Constructor University Bremen, Campus Ring 1, 28759 Bremen, Germany.

Correspondence: `XXX@constructor.university`

© 2025 Constructor University Mathematics Society. All rights reserved.

About This Journal

The Journal

The *Constructor Mathematical Journal* (**CMJ**) is the undergraduate mathematics journal of Constructor University Bremen, published annually by the Constructor University Mathematics Society (CUMS).

CMJ publishes expository and research articles written by undergraduate and graduate students. All articles originate in the CUMS Mathematics Undergraduate Seminar and are developed into self-contained expositions accessible to a broad mathematical audience.

Editorial Board

Editor-in-Chief
XXX — Name *(Constructor University)*

Managing Editor
XXX — Name *(Constructor University)*

Associate Editors
XXX — Name *(Constructor University)*
XXX — Name *(Constructor University)*

Faculty Advisor
XXX — Name *(Constructor University)*

Contact

Constructor University Mathematics Society
Campus Ring 1, 28759 Bremen, Germany

`XXX@constructor.university`
`XXX -- math.constructor.university`

Submission Guidelines

We welcome submissions from students at Constructor University and from visiting researchers who have participated in the CUMS seminar. Manuscripts should:

- be written in LaTeX using the `cumsj` style package (available at the journal website);
- include an abstract of 80–150 words;
- be self-contained and aimed at an undergraduate audience;
- include a complete reference list.

Submissions and enquiries may be sent to the editorial address above. The journal website provides the article template, style guide, and submission portal:

`XXX -- math.constructor.university/cmj`

Recorded Talks

Each article in this volume is accompanied by a recorded talk delivered at the CUMS Mathematics Undergraduate Seminar. The recordings are freely available and provide an accessible entry point to each paper's ideas:

`XXX -- math.constructor.university/talks`

The talks are typically 30–45 minutes and are aimed at students with a solid background in first-year undergraduate mathematics. All recordings are hosted on the university's streaming platform and may be accessed without registration.

Acknowledgements

The editors thank the Constructor University Mathematics Department for their generous support, and all seminar participants whose questions and discussions shaped the final articles.

Contents

Shannon's Theorem and Polar Codes 1

Aly Assaf

The Qubit and Shor's Algorithm 16

Ahmed Maghri

Bragg's Law and X-Ray Crystallography 30

XXX — Author Name

Shannon's Theorem and Polar Codes

Aly Assaf

Constructor University Bremen

Abstract. Coding Theory is responsible for facilitating reliable message-passing over communication channels. In 1948, Claude Shannon proved that for every channel there exists a family of error-correcting codes that allow reliable communication at any rate below the channel's capacity. Shannon's work is, however, non-constructive, as it provides no method to design such efficient codes. While there have been attempts over the years to reach this theoretical capacity, notably with LDPC codes (1990) and Turbo Codes (1993), these could only empirically approach the theoretical limit. This stood for 60 years until Erdal Arıkan's introduction of Polar Codes, the first clean and mathematically proven family of codes to achieve Shannon's maximum capacity over symmetric binary-input memoryless channels. This expository paper aims to briefly introduce the general Coding Theory scheme, outline the key ideas in Shannon's arguments, and explore Arıkan's outstanding construction of polar codes.

1 Coding Theory Fundamentals

Before proceeding we need to build a framework of definitions and the general idea behind Coding Theory. This paper is designed such that anyone with a basic background in mathematics can understand. The details of what information and entropy are have been moved under the section 'Shannon's Theorem' for convenience as they follow directly into his idea of relating channel capacity with information rate of a code.

In real life, messages are sent as a block of symbols through some medium, aka a 'channel'. For example, binary code.

Channel: a formal mathematical model of communication that operates abstractly like a function, in the sense that it has input and output sets. Channels apply a probabilistic function on each symbol of the input to produce the output. The set of all symbols is the alphabet Σ

In our case we use the Binary Symmetric Channel.

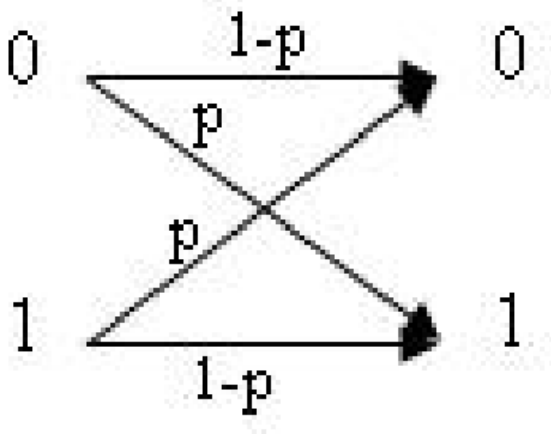


Figure 1: Model of a BSC_p

Binary Symmetric Channel (BSC): both input and output sets consist of the alphabet $\Sigma = \{1, 0\}$. The probabalistic function is the Bernoulli random variable with probability p of being flipped as visualised in Figure 1 and Equation 1. The Bernoulli random variable is hereafter denoted as as $X \sim Bern(p)$.

$$u = \begin{cases} u & 1-p \\ u' & p \end{cases} \tag{1}$$

A more generalized channel is also the discrete memoryless channel. However, we will not be discussing it in this paper. **Discrete Memoryless Channel:** Channels that transmit sets of symbols from a finite alphabet Σ. *Memoryless* simply means that the probabilistic function is independent of messages sent before the current message.

Now that errors have been modeled, we must find a way to correct or detect them. Here we introduce the notion of codes through the below definitions.

error-correcting code: an algorithm that represents messages of length k as codewords of length n where $n > k$ such that the occurance of errors after transmission can be detected or corrected.

message: a sequence of symbols $\mathbf{u} = u_1 u_2 ... u_k$. It is the information we desire to transmit.

codeword: A sequence of symbols $\mathbf{x} = x_1 x_2 ... x_n$ where each codeword corresponds directly to a message $\mathbf{u}$ as $\mathbf{x} = \mathbf{x}(\mathbf{u})$.

The set of all messages and codewords are U and χ, where $|\chi| = |U| = 2^k$

More specifically, we are looking at Linear Codes, the parent family of Polar Codes.

x_1	x_2	x_1 XOR x_2
0	0	0
0	1	1
1	0	1
1	1	0

Figure 2: XOR truth table

1.1 Definitions and Properties of linear codes

Definition 1: A code is linear if there exists a matrix H such that $Hx^T = 0 \quad \forall u$. This is called the parity check matrix.

(a) Additionally, $H(x+y)^T = Hx^T + Hy^T = 0$ and so $x+y$ is also considered a codeword. This is the linearity property of linear codes.

Definition 2: For every linear code there exists a k x n generator matrix G such that $\mathbf{x} = \mathbf{u}G$ and $GH^T = HG^T = 0$.

Definition 3: The information rate (or efficiency) of a code is $R = \frac{k}{n}$.

The matrices and properties above can be used directly to encode and decode simple codes via the H and G matrix pair.

Definition 4: The Hamming weight is the count of '1's present in a base 2 vector. It is denoted by $wt(\mathbf{x})$

Definition 5: The Hamming distance between any 2 vectors $\mathbf{x}$ and $\mathbf{y}$ is defined as the number of differences between them. The notation is as follows: $dist(\mathbf{x}, \mathbf{y}) = wt(\mathbf{x} - \mathbf{y})$

Definition 6: The minimum distance of a code is the minimum Hamming distance between any two codewords. It is often parameterized as d. This is simply referred to as the *distance* of a code.

Definition 7: the XOR operation between any two bits is denoted X + Y where X and Y are bits, and is defined in Figure 2

Finally, we can describe simple binary codes as a set of parameters $[n, k, d]$, encapsulating the message and codewords size, alongside the minimum distance between each codeword.

To better understand the minimum hamming distance of a code, recall that the size of the set of

codewords and messages are equal, yet the length of each codeword is longer than the original message. This means the set of codewords $\chi \subset \{0,1\}^k$, leaving gaps in between.

We can visualize this using Hamming balls in Figure 3. The center represents a valid codeword and the radius outlines the area containing other elements of $\{0,1\}^k$ that aren't codewords. These are the elements obtained due to some error during transmission.

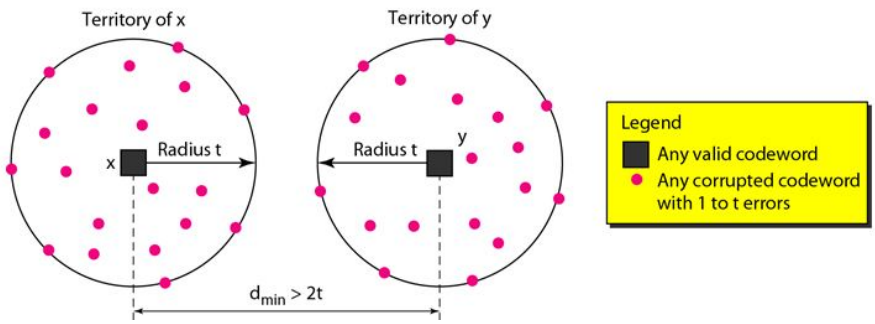


Figure 3: Visualization of Hamming balls with radius t and minimum distance of 2t. x and y are both codewords

Encoding is a simple process of just applying the generator matrix. We now explore one simple decoding method that will also provide insight into what the minimum distance truly means and its importance in maintaining a stable code.

1.2 minimum distance decoding

Given a codeword x sent over a BSC such that the received codeword is y and probability of error is $0 \leq p < \frac{1}{2}$, the error vector is $e = x - y$. It describes the differences between sent and received codewords. The number of errors, aka the weight of e, is denoted a. In an ideal case, for example, e would be a zeros vector with $a = 0$.

We can find the probability of each error as $Pr(e = w) = p^a(1-p)^{n-a}$. Where w is some error vector.

By observing the pattern as a increases, we see that the most probable error vector (that is non-zero) is one with the least number of errors. As $(1-p)^n > p(1-p)^{n-1} > p^2(1-p)^{n-2} > \ldots$ due to our bounds on p, we can see that the most probable error vector (that is non-zero) is one with the least number of errors. Using this, we can test all 2^k error vectors in order from smallest to greatest error until we find a value for x that is within the set of all valid codewords χ.

In minimum distance decoding, the radius of the hamming ball is the maximum number of errors that can occur and still be corrected. If there is an overlap between hamming balls, then there is the possibility of an error occurring where it could be corrected to either valid codeword. By introducing redundancy, we increase the minimum distance to reduce this overlap, but as a result reduce the efficiency of the code.

For example, take codewords c_1 and c_2. if the distance between them is 1, we can only detect an error of distance 1 using the parity matrix, but cannot correct it with certainty. This is

because we cannot determine which codeword the error originated from. However, if we have a distance of 2, we can detect errors up to distance 2 and we can correct error up to a distance of 1. This can be visualized as hamming balls with a distance between codewords of 2, and radius of 1, resulting in no overlap and is thus stable.

1.3 Extra Mathematics

The following mathematical ideas will help in our understanding of Shannon's Theorem but are not essential for the paper.

1.3.1 Law of Large Numbers (LLN)

The LLN states that as the sample size (n) of a random variable increases, the sample mean converges to the expected value (population mean).

Let $X_1, X_2, \ldots, X_n$ be independent and identically distributed (i.i.d.) random variables with finite mean $\mu = \mathbb{E}[X_i]$. The average is

$$\overline{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i.$$

Weak Law of Large Numbers: For any $\epsilon > 0$,

$$\Pr\left(\left|\overline{X}_n - \mu\right| > \epsilon\right) \to 0 \quad \text{as } n \to \infty.$$

This means that the sample average converges to μ in probability.

1.3.2 Chernoff Bounds

Chernoff bounds help us find the probability of obtaining a large value in a random variable.

Imagine you are flipping a biased coin n times. You expect to see np heads, but in reality it could be within some range of np.

Chernoff bounds state that the probability of obtaining a value far from np decreases exponentially as n increases.

This is different from the Law of Large numbers, which would instead state that the fraction of heads to tails would approach p.

This probability is formally defined as:

$$P(X \geq a) \leq e^{-at} M_x(t) \qquad \forall t > 0$$

Where $M_x(t)$ is the Moment Generating Function and $a = (1+\delta)\mu$ as seen in 4.

2 Examples of Linear Codes

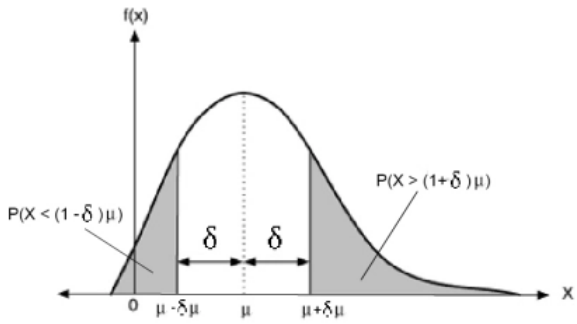


Figure 4: Probability distribution of a random variable

3 The Coding Problem

The coding problem is the challenge of designing codes that can efficiently and reliably transmit data over noisy channels. To do this, we try to maximize the rate of transfer R which involves increasing k and reducing n while simultaneously increasing the distance d to allow us to correct more errors.

3.1 Example

Let's give a simple example of us encoding and decoding by talking. We define a Discrete Memoryless Channel (DMC) where the alphabet is the entire dictionary, so every word is a symbol. Then we define our noise function as the following: Every symbol has a probability let's say $p = \frac{1}{4}$ to have an error, meaning that symbol would be replaced by a random other symbol (the distribution does not matter here as this is not formal and is purely for your understanding). For example, if the word 'play' was rolled as an error, it could become 'exfoliating' or 'mouse' or 'summer', etc.

Now given this context, Bob wants to tell Alice 'wanna play basketball' over this channel, but Alice could hear 'never play basketball'. How can we correct or detect the error? One way is by using a repetition code. This involves sending multiple copies of each symbol, and the receiving end would choose the symbol that appears most commonly in each group of 3. Now Alice hears: 'wanna never wanna play play play basketball football basketball', she decodes the message and hears 'wanna play basketball'. No more errors! However, notice this adds redundancy to the data we're sending, which makes it far less efficient. If our channel becomes noisier, let's say $p \geq \frac{1}{3}$ then our code will almost never work and we'd need to add more redundancy by sending more copies to ensure transmission is reliable enough.

Most engineers tinkered and tried to create the ideal code by varying the ratio of information bits to redundant bits and testing the reliability, but Shannon was the only one to ask 'What is the thoeretical maximum rate of information I can have whilst still being reliable?'.

3.2 Exploring Reliability

The reasoning of why maximizing R increases our rate of transfer is trivial, however, why does increasing the distance improve the reliability of the code? To explore this idea, let us select a random encoding and decoding function, E and D for every message $\mathbf{u}$ of length k. The codeword $E(u)$ is chosen randomly from the set of all codewords with uniform probabilities. The decoding function is maximum likelihood decoding, which is also known as minimum distance decoding outlined in the preliminaries section. Please follow alongside Figure 5 for the following section.

Suppose we send codeword $E(u)$ and it is received on the other end as y. Now draw a Hamming ball with radius pn - the expected number of bit flips after transmission - and center at $E(u)$. Observe Figure 5 for a visualization of what we are doing. By Chernoff Bounds, we know that y is almost certainly within our ball.

Next, draw another ball around y of the same radius pn. The only scenario where our decoding function could fail is when there is another codeword $E(u')$ that is within a ball of radius pn around y.

As we increase k, we also increase the number of valid codewords. This increase in density reduces the minimum distance.

Here we are viewing the same problem of trading off efficiency for reliability from a different perspective.

Note: you can imagine y drifting away from the original codeword as it's being sent, then once received, it is matched with the closest codeword and decoded, this is the principle of what is happening above.

4 Shannon's Theorem

Following from our previous discussion, we can see that this is the challenge we face in Coding Theory; how do we balance reliability with efficiency? Is it possible to create a perfectly balanced code?. In communication systems, engineers had the belief that errors during transmission were inevitable, that at most we can minimize the rate of error. Keep in mind this is not speaking in terms of the previously mentioned error probability p, rather, the error rate in this case is the difference between the original message and the final decoded message. The main question posed was: 'Is it possible to reliably transmit information over noisy channels, and if so, how quickly?'.

This is where we would like to introduce the notion of *capacity*. But before this we must build an understanding of what are *information* and *entropy*.

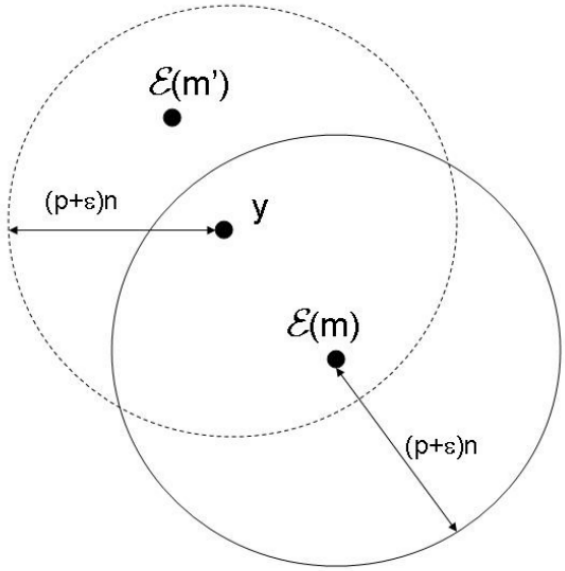


Figure 5: Hamming balls with received message y and sent message $E(x)$

4.1 Information

In information theory, the amount of information we get from an event is defined as

$$I(X = x) = \log\left(\frac{1}{P(x)}\right) \tag{2}$$

where $P(x)$ is the probability of event x. This quantity can be though of intuitively as the 'surprise factor'; how surprised are you of this outcome? Trivially, if the probability of an event is almost certain, your surprise factor would be 0 and vice versa as observed in I. The base of the log determines the unit, either nats for base e or bits for base 2. For our context, we will be using base 2 hereafter.

4.2 Entropy

Entropy of a random variable is defined as

$$H(X) = \sum_{x \in X} P(X = x) \log\left(\frac{1}{P(X = x)}\right) \tag{3}$$

Comparing this with the way we obtain the average of a random variable below:

$$\mu_x = \sum x \cdot P(x)$$

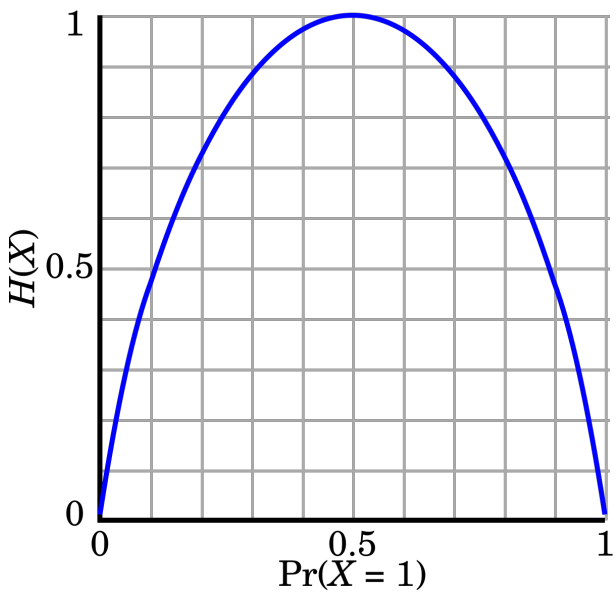


Figure 6: Entropy of a BSC

It can be intuitively understood that Entropy is the average information you get from a random variable. The more distributed the probability function, the greater the average information. It can also be thought of non-mathematically as the *randomness*.

In the case of a BSC, the plot of entropy to probability is seen in Figure 6.

For example, say you flip a fair coin once. The two events are:

$$P(H) = 0.5, \quad P(T) = 0.5$$

The entropy is:

$$H(X) = -\left[0.5\log_2 0.5 + 0.5\log_2 0.5\right] = -\left[0.5(-1) + 0.5(-1)\right] = 1 \text{ bit.}$$

Thus, each fair coin toss carries exactly 1 bit of information.

If the coin is biased, say

$$P(H) = 0.9, \quad P(T) = 0.1,$$

then the entropy is:

$$H(X) = -\left[0.9\log_2 0.9 + 0.1\log_2 0.1\right] \approx 0.47 \text{ bits.}$$

So, the biased coin has less uncertainty, hence lower entropy.

4.3 Mutual Information

An abstract concept following naturally from entropy is mutual information or 'how much more information do I have about random variable X if I am given Y if they depend on eachother'. Mutual information is denoted:

$$I(X;Y) = H(X) - H(X|Y) \tag{4}$$

Where $H(X|Y)$ is defined as:

$$H(X \mid Y) = -\sum_{y \in \mathcal{Y}} P(y) \sum_{x \in \mathcal{X}} P(x \mid y) \log_2 P(x \mid y). \tag{5}$$

4.3.1 Example with Perfect Dependence

An example will provide you with better insight and intuition on mutual information.

Consider again the fair coin toss X from the previous example regarding entropy. Let

$$X = \text{Toss of a fair coin (H/T)}, \quad Y = \text{“Is the coin heads?” (Yes/No)}.$$

If you know X, then you know Y. If you know Y, you also know X.

The entropies are:

$$H(X) = 1 \text{ bit}, \quad H(Y) = 1 \text{ bit}.$$

The conditional entropy is:

$$H(Y \mid X) = 0.$$

because given X, you already know Y with certainty.

Therefore, the mutual information is:

$$I(X;Y) = H(Y) - H(Y \mid X) = 1 - 0 = 1 \text{ bit}.$$

This means if you measure X, you get 1 bit of information about Y and vice versa.

A nice visual of entropy in terms of a venn diagram is available in Figure 7.

4.4 Capacity

The abstract definition of capacity follows from the idea of mutual information.

Definition 8: Capacity is the maximum amount of information transmitted per channel use. Mathematically, it is the maximum mutual information we have on sent message x given received message y: $max \;\; I(X;Y)$

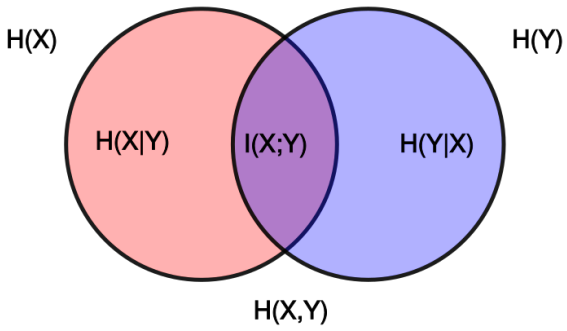


Figure 7: Venn diagram representation of entropy and information

In 1948, Claude Shannon wrote the theorem relating channel capacity with the information rate of a channel, proving that it is possible to have completely error-free communication in discrete memoryless channels so long as the information rate $R < C$ where C is the 'channel capacity'.

Theorem 1.1: $\forall$ binary symmetric channels with $0 \leq p < \frac{1}{2}$, $\exists$ channel capacity C s.t all codewords $\mathbf{x}$ can always be reliably decoded into their original message $\mathbf{u}$ with arbitrarily low error probability Pr_e so long as $R < C$.

In the case of binary symmetric channels, maximum capacity is defined as: $C = 1 - H(p) = 1 + p\log_2 p + (1-p)\log_2(1-p)$ .Pr_e tends to at most $2^{-\sigma n}$. where $\sigma = 1 - H(p) - R$, aka the difference between maximum and actual rate So as $n \to \infty$, the error rate approaches 0.

However, one may ask, won't the error rate approach 0 regardless of the value of R? This is a question I thought to myself and the answer is very clearly no. Assume $R > C$, then $\sigma < 0$, and so Pr_e would diverge as n increases. And so the converse of the theorem is presented where, in the case $R > C$, reliable communication is impossible.
While Shannon's theorem was revolutionary, it unfortunately did not provide an empirical method to creating such efficient codes. However, by knowing this limit, engineers and mathematicians alike were able to evaluate and design their codes to be as efficient as possible while still being reliable. Examples of this include Turbo Codes (1993), LDPC codes (1990), and now Polar Codes (2008).

To feed our curiosity, let us present a very brief overview of the strategy of a geometric proof to feed the reader's curiosity and provide a general intuitive understanding. This proof is for the case of binary symmetric channels.

Take a randomly generated linear code $\mathbf{C}$ with some encoding and decoding function. Efficiency does not matter. Our decoding function will be minimum distance decoding. Let's return to our hamming space visualization in 5 and recall our explanation of error as the *drift* of the

output within its respective hamming ball. By the law of large numbers, the received codeword is almost surely going to be within pn distance of the original codeword.

The number of possible sequences with exactly pn flips using Stirling's approximation is

$$\binom{n}{pn} \approx 2^{nH(p)}.$$

Since the total number of binary sequences of length n is 2^n, the maximum number of disjoint Hamming balls you can pack is approximately

$$\frac{2^n}{2^{nH(p)}} = 2^{n(1-H(p))}.$$

This shows that reliable communication is possible only if the code rate satisfies

$$R < 1 - H(p).$$

5 Polar Codes

5.1 Channel Polarization - The Idea

Polar Codes are a family of linear codes that are the first family of codes to reach the Shannon Limit aka 'channel capacity'. They were created by Arikan in 2008 and later mathematically proven to reach the limit in 2013.

Arikan initially had the goal of finding a way to compress a message u that is n bits long. He paired up each bit in the input message as (U, V). He tried encoding the bits by simply performing the XOR operation between the two bits, $(U, V) \to U \oplus V$, however, one bit of information was lost as a result. Therefore, he added another value that is linear and different from $U \oplus V$, simply choosing a copy of the second bit. So far, Arikan has encoded, $(U, V) \to (U \oplus V, V)$. Given the pair $U \oplus V$ and V, the operation is easily reversible, and therefore no information is lost.

One crucial property Arikan noticed is that, although he didn't succeed in compressing the information, he encoded it in a way that altered the entropy of each bit-channel (meaning each bit now has it's own probability and channel capacity). But how do the entropies of each new bit-channel change?

Statement 1: Given U and V are independant Bernoulli random variables with $0 \leq p \leq \frac{1}{2}$, we want to show that for $U \oplus V$, the new probability of error p' is greater than p.

Given that U and V are mutually exclusive independant events, we can compute p' like so:

$$p' = \mathbb{P}(U \oplus V = 1) = \underbrace{\mathbb{P}(U = 1, V = 0)}_{p(1-p)} + \overbrace{\mathbb{P}(U = 0, V = 1)}^{(1-p)p} = 2p(1-p).$$

Then, the difference $p' - p = \overbrace{p}^{\leq \frac{1}{2}}(1 - \underbrace{2p}_{\leq 1})$ is a positive quantity. This asserts the claim. Next,

Statement 2:

a) The first sub-channel's entropy increases: $H(U \oplus V) > H(U)$ and $H(U \oplus V) > H(V)$.

b) The second sub-channel's entropy decreases: $H(V \mid U \oplus V) < H(U)$ and $H(V \mid U \oplus V) < H(V)$.

From now on we denote $h(p)$ as the entropy of a Bernoulli random variable with parameter p. The first statement is trivial; $p' > p$ implies $h(p') > h(p)$. For the second statement, recall that the joint entropy of two independent random variables is the sum of their entropies:

$$H(U, V) = H(U) + H(V)$$

where $H(U, V)$ is the joint entropy of both U and V. After Arıkan's transformation, we have the new pair $(U \oplus V, V)$. It is reversible to (U, V), and so $H(U \oplus V, V)$ should equal $H(U, V)$. The chain rule of entropy yields

$$\underbrace{H(U \oplus V, V)}_{=2h(p)} = \overbrace{H(U \oplus V)}^{=h(p')} + H(V \mid U \oplus V)$$

[1] As a result, we can see

$$H(V|U \oplus V) = 2h(p) - h(p') < h(p)$$

As a final result, we observe how elements in the first group have entropy $H(U \oplus V) > h(p)$ and elements of the second sub-channel $H(V|U \oplus V) < h(p)$. So the first sub-channel has a larger entropy, and so its channel capacity decreases, and vice versa for the second channel. This manipulation of channel capacities is called *channel polarization* - see Figure **??**, and it's the key idea behind Polar Codes.

[1] Can you figure out why $H(U \oplus V, V) = 2h(p)$?

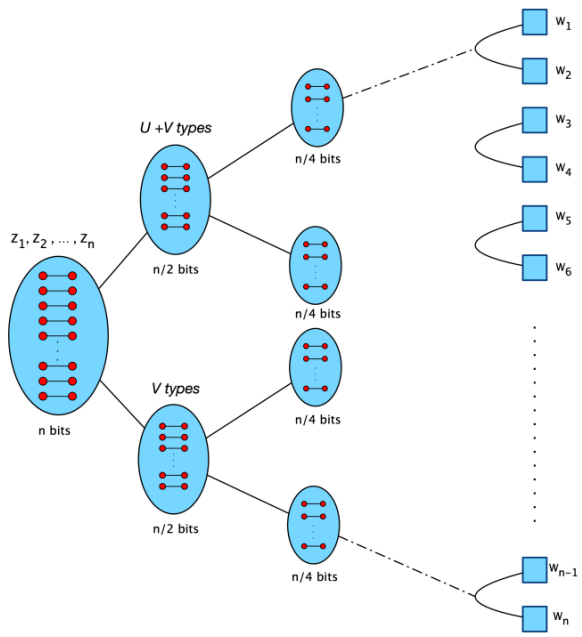
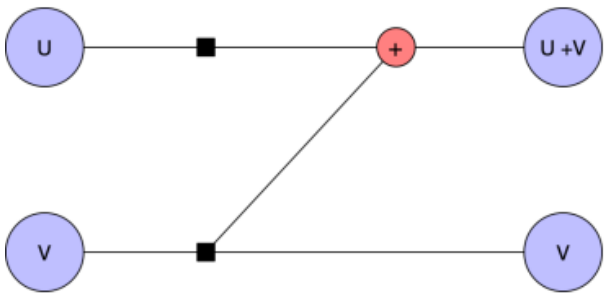


Figure 8: Polarization process



Consider what would happen if we recursively applied this algorithm. How would the channel entropies change? Let us take 2^n copies of the BSC, called subchannels, and recursively apply the above. We pair up U and V, apply the operation, then separate $U \oplus V$ objects and V objects whilst maintaining their order. Repeat this until only n individual bits remain after separation that have either entropy 1 (useless) or entropy 0 (ideal). Observe Figure 8 for a visualization.

The final result of this stage is a set of n virtual channels, either with channel capacity 0, 1 or somewhere in between. We select the channels with capacity 1 to send data through and ones with capacity 0 are *frozen*, meaning they are set to 0 and do not store meaningful data.

The careful readers may notice a glaring issue; won't there still be some 'middling' channels that have a mediocre channel capacity of 0.3, 0.6, 0.7, etc. Fortunately, Arikan proved in his paper that as number of bits $N \to \infty$, the fraction of 'middling' virtual channels vanishes, leaving channels with either channel capacity of 0 or 1.

Unfortunately, for a small number of bits, the polarization process results in some virtual channels having a mediocre capacity - 0.3, 0.6, 0.8.

5.2 Touching on Implementation and Advantages

As the operations are all reversible, decoding it is a simple but long algorithm. If we observe Figure 8 (the recursive polarization) we can see that w_n is exclusively a V-type and does not depend on another bit channel, so it can be used to decode w_{n-1}, then w_{n-1} can be used to decode w_{n-2} and so on in a process called successive cancellation decoding which is outside the scope of this paper.

The properties of polar codes make them suitable An advantage of polar codes is that the number of steps to fully polarize the channels is a known value $\log_2 n$, since the number of copies halves at each stage.

6 Conclusion & Final Remarks

We've followed in the footsteps of one of the greatest revolutionaries that built the foundations of reliable and efficient communication—Claude Shannon. By exploring coding and information theory, we not only built our understanding on a number of underlying concepts but also developed a bird's eye view on how Arıkan took the teachings of Shannon to a practical application to the benefit of humanity. The ingenuity and experimentation that Arıkan expressed during his development of Polar Codes is an impressive and satisfying learning experience.

Shannon's Theorem is a beautiful mathematical construct that creates a clear guideline for engineers to build on. Polar Codes are the practical realisation of this. They are an incredible tool in the engineer's arsenal that allows optimised communication under the correct circumstances.

References

[1] P. Elias, "Error-Free Coding," *IRE Trans. Inform. Theory*, vol. 4, no. 4, pp. 29–37, 1954.

[2] V. Guruswami and A. Rudra, "Proof of Shannon's Theorem and an Explicit Code," Lecture 4, *CSE 533: Error-Correcting Codes*, Carnegie Mellon University, Oct. 2006.

[3] V. Guruswami and D. Halperin, "More on Linear Codes," Lecture 2, *CSE 533: Error-Correcting Codes*, Carnegie Mellon University, Oct. 2006.

[4] V. Guruswami and W. Machmouchi, "Shannon's Theorem," Lecture 3, *CSE 533: Error-Correcting Codes*, Carnegie Mellon University, Oct. 2006.

[5] V. Guruswami and V. Rastogi, "Error Correcting Codes," Lecture 1, *CSE 533: Error-Correcting Codes*, Carnegie Mellon University, Oct. 2006.

[6] L. Slot and S. Zur, "Shannon's Noisy-Channel Coding Theorem," Vrije Universiteit Amsterdam, Feb. 2015.

[7] M. Goemans, "Shannon's Noisy Coding Theorem," 18.310 Lecture Notes, MIT, May 2015.

[8] S. Biswas, "Introduction to Coding Theory: Basic Codes and Shannon's Theorem," Undergraduate Research Paper, UChicago, 2011.

[9] M. Sudan and M. Suzgun, "A Gentle Introduction to Polar Codes," Lecture 8, *CS 229r: Information Theory in Computer Science*, Harvard University, Jan. 2019.

[10] O. Elshinawy, "Syllabus & Agenda: Mathematics Undergraduate Seminar," Spring 2025.

The Qubit and Shor's Algorithm: A Step-by-Step Exploration

Ahmed Maghri

Constructor University Bremen

Abstract. Shor's algorithm presents a polynomial-time quantum solution to the integer factorisation problem, threatening classical encryption protocols like RSA. This article introduces the mathematical and computational foundations required to understand the algorithm, including modular arithmetic, continued fractions, and quantum principles. We provide a step-by-step breakdown of Shor's algorithm and its quantum subroutine, emphasising accessibility and conceptual clarity, while also addressing practical constraints in current quantum hardware. This article aims to make the concepts of Shor's algorithm accessible to readers with limited background in mathematics and quantum computing, while maintaining mathematical rigour and conceptual depth throughout.

Acknowledgements. I would like to sincerely thank Omar Elshinawy, head of the Mathematics Society at Constructor University, for his valuable guidance and support throughout the research and writing of this article. I also wish to thank Muaz Hussein, a first-year ECE major, for providing the first reviews and constructive feedback on the manuscript. A heartfelt thanks goes to my mathematics and computer science professors and fellow Math Society members, whose enthusiasm and insight made this research journey both enriching and enjoyable. Finally, and most importantly, I express my deepest gratitude to my parents, without whom none of this would have been possible.

0 Preliminaries.

This article has been made such that almost any person is able to understand the whole content, however, a minimum of reasoning, computer science, and mathematics concept will be required, and to ensure that the reader has these requirements, here are some definitions and concepts that we are going to use during our journey (and even some advanced definitions). The reader is not obliged to read it now, but whenever he needs to.

1- Number's Theory:

- **Modular arithmetic:** We say $A \equiv B \pmod{N}$, when the division of A by N gives a remainder of B. Quick example: $\frac{5}{2}$ gives a remainder of 1 $(5 = 2 \times 2 + 1)$. Thus we say: $5 \equiv 1 \pmod{2}$. If this is still a bit unclear for you, let's say that you take the number A and subtract N until you can't. In our previous example:

$$5 - 2 = 3, \quad 3 - 2 = 1$$

  The last number is the remainder.

- **A is a divisor of B:** A is able to divide B, A/B gives an integer.

- **The gcd of A and B (gcd(A, B)):** It's the greatest common divisor of A and B. For instance: 24 and 18 have as common divisors: 1, 2, 3, 6. So $\gcd(24, 18) = 6$.

- **A and B are coprime:** $\gcd(A, B) = 1$. Meaning that A and B have no common divisors (not counting the trivial 1, which is a divisor of all whole numbers). For example, $\gcd(15, 8) = 1$, so 15 and 18 are coprime.

- **The order of A modulo N:** is the smallest number r such that $A^r \equiv 1 \pmod{N}$.

- **A is semiprime:** A just has two divisors (not counting 1 and A itself). For example, 15 is semiprime. Another way to see it is that A is the product of two prime numbers.

2– Computer Science:

- **Polynomial time:** We say a problem can be solved in polynomial time if: Given a problem of size n, the problem is solvable in a time of n^r, with r a real number.

- **P problems:** Are the problems for which we can find solutions in polynomial time.

- **NP problems:** Problems for which we can't find solutions in polynomial time. Also explained as problems where we can easily check if a given solution is correct, but finding that solution is very hard.

- **An algorithm:** A set of instructions that is designed to accomplish a task.

- **A private key:** A key that is known only by its owner.

- **A public key:** A key that is known by everyone.

- **Converting binary to decimal:** Let's say 1101 is our binary number. You compute:

$$1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 13$$

  This helps illustrate the pattern. If not clear, a quick Google search will explain it very well.

- **Number of possible states:** Something we will use often: if we have n bits, then we have 2^n possible states.

- **BQP problems:** BQP stands for Bounded-Error Quantum Polynomial Time. These are decision problems that can be solved by a quantum computer in polynomial time with a probability of at least $\frac{2}{3}$ of being correct. The error probability is at most $\frac{1}{3}$ for all instances.

3– Computer Architecture:

- **A bit:** A bit (short for binary digit) is the smallest unit of information in classical computing. It can take on one of two possible values: 0 or 1.
- **A register:** A register is a small, fast storage location inside a computer's processor that holds data temporarily. In classical computing, it typically consists of several bits and is used for arithmetic or logical operations.

4– Linear Algebra:

- i**:** Represents the elementary complex number, where $i^2 = -1$.
- **Transpose of a matrix:** Given a matrix M, its transpose is the matrix with rows and columns swapped. For example, here is a matrix and its transpose:

$$M = \begin{bmatrix} 1 & 2 \\ 3 & 3i \end{bmatrix}, \quad M^T = \begin{bmatrix} 1 & 3 \\ 2 & 3i \end{bmatrix}$$

- **Complex conjugate of a matrix:** Given a matrix M with complex entries, its complex conjugate is basically the same matrix but negating each element that has i in it:

$$M = \begin{bmatrix} 1 & 2 \\ 3 & 3i \end{bmatrix}, \quad \overline{M} = \begin{bmatrix} 1 & 2 \\ 3 & -3i \end{bmatrix}$$

5– Quantum Computing:

- **Physical qubits:** Physical qubits are the actual hardware qubits built using physical systems (like trapped ions or superconducting circuits).
- **Logical qubits:** Idealized qubits, error-corrected, essentially used in theory. Approximately 1 logical qubit $\approx$ 10,000 physical qubits.
- **Transformation to matrix:** Each quantum transformation can be represented as a matrix, or as its direct application on a qubit (for linear algebra adepts, you can see it as when you can represent a transformation in its matrix form or directly how it acts on the vectors).

6– Set's Theory:

- $\mathbb{N}$**:** Natural numbers — counting numbers like 1, 2, 3, ...
- $\mathbb{Z}$**:** Integers — all whole numbers, positive and negative, including zero: $\ldots, -2, -1, 0, 1, 2, \ldots$

- $\mathbb{Q}$: Rational numbers — numbers that can be written as a fraction $\frac{a}{b}$ where $a, b \in \mathbb{Z}$ and $b \neq 0$
- $\mathbb{R}$: Real numbers — all rational and irrational numbers (e.g., π, $\sqrt{2}$)
- $\mathbb{C}$: Complex numbers — numbers of the form $a + bi$, where $a, b \in \mathbb{R}$ and $i^2 = -1$

1 Introduction to Cryptography.

Cryptography is the study of mathematical techniques for securing communication and data in the presence of adversaries. Its primary goal is to ensure the confidentiality, integrity, authenticity, and non-repudiation of information. We are mainly dealing with two subjects in it: Encryption & Decryption.

Definition 0:

In cryptography, **encryption** (more specifically, **encoding**) is the process of transforming information in a way that, ideally, only authorized parties can decode.

1.1 RSA.

The RSA cryptosystem was introduced in 1977 by **Rivest**, **Shamir**, and **Adleman**, from whom it takes its name.I t is a public-key encryption scheme designed to securely encode messages over insecure communication channels by relying on the computational difficulty of factoring large composite integers. Let's discuss how it works and what is the algorithm behind it.

1.2 Algorithm of RSA.

Take Alice and Bob. Alice and Bob both have two big prime numbers (in general around 313 digits). Let's name the prime numbers of Alice P1 and P2, and Bob's P3 and P4.

Both Alice and Bob are going to multiply their two big prime numbers to get a new big semiprime number. Let's call them respectively SP1 and SP2.

P1 and P2 are then called the private keys of Alice, SR1 is her public key. And similarly for Bob.

If Alice wants to send something to Bob, she is going to hash her message in a way such that she uses his public key SP2, and the only way to decrypt her message, must be that her receptor knows the two big prime numbers P3 and P4, that created the public key SP2. In our case, Bob knows them, but no one else knows does. Both the public keys of Alice and Bob can be viewed by anyone, as long as her hashed message. However, due to the fact that only Bob knows what P3 and P4 are, he is the only one able to decode the hashed message.

Note 0:

That we are not covering how the message is hashed, and we won't. This part is not relevant for the rest of the article and thus is out of scope.

Note 1:

We call this an asymmetric key system because two different keys are used to send and to receive the message.

1.3 How to make RSA irrelevant?

The whole RSA is standing on one assumption: Finding the prime factors of a semiprime number is a NP problem. Meaning that if you have a big semiprime Number N, I can check if two prime numbers are its two prime factors (divisors). However, finding the two prime factors takes way more time, it is possible, but for example for a 313 digits semiprime number, it may take 24000 years to find them, even for a supercomputer. Thus, to be able to make RSA breakable, it is enough to show that the problem of finding the prime factors of a semiprime number is actually a P problem, so if we find a way to find efficiently factor a semiprime number, we broke the RSA system.

And this is where Shor's algorithm comes.

1.4 Introduction to Shor's Algorithm.

Shor's algorithm, developed by **Peter Shor in 1994**, is a quantum algorithm for efficiently factoring large integers. It runs in polynomial time on a quantum computer and poses a fundamental threat to classical cryptographic systems such as RSA, whose security relies on the hardness of integer factorization.

But before explaining how it works, you may have noticed that there is a quite scary word in its definition, "quantum". Thus, before explaining the algorithm and how it isg implemented, for the sake of the reader's comfort, I will have to give you a minimum knowledge about the world of Quantum Computing.

2 Introduction to the world of quantum computing.

Note 2:

Our goal in this article is not to focus on quantum computing, there are multiple lectures about it, and some really advanced topics will be needed during section 3.3 to explain the implementation of Shor's algorithm, and these concepts need their own article. If you feel a bit lost during the quantum computing parts, this is totally normal, but I am going to do my best to keep you on board.

Note 3:

While reading this article, you may encounter some intermediary algorithms, while the article will still give a short explanation of them, again our goal won't be to go deeply into them. If you would like to know more about them, a quick google search will serve you.

2.1 Theoritcal part.

Briefly: Quantum computers use quantum bits (qubits) instead of normal bits.

We all heard this, without really getting what is happening behind it. We also heard the famous sentence: While bits must be either in a 0 or 1 state, a qubit can be in a superposition of states $|0\rangle$ and $|1\rangle$. Some people also heard that when you measure a quantum state, you can't measure the superposition, you just randomly measure one of the possible states. At measurement, you can say that the quantum states collapse into exactly one. Let's try to discover what is really happening behind the scenes.

Formally, a quantum state (so a state "between" the $|0\rangle$ and the $|1\rangle$) is described by amplitudes α and β, with probability $|\alpha|^2$ of measuring 0, and probability $|\beta|^2$ of measuring 1. Thus, we have:

$$|\alpha|^2 + |\beta|^2 = 1$$

And we write for a state X:

$$|X\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$$

For a state with 2 qubits, instead of 1, we get 2^2 possibilities:

$$|X\rangle = \alpha\,|00\rangle + \beta\,|01\rangle + \gamma\,|10\rangle + \delta\,|11\rangle$$

$$|\alpha|^2 + |\beta|^2 + |\gamma|^2 + |\delta|^2 = 1$$

As an example, consider the state Y where:

$$|Y\rangle = \frac{1}{2}\,|00\rangle + \frac{1}{2}\,|01\rangle + \frac{1}{2}\,|10\rangle + \frac{1}{2}\,|11\rangle$$

If we try to measure the state Y, we have equally $\frac{1}{4}$ probability of measuring either 00, 01, 10, or 11.

2.2 Physical part.

But how do we create such a computer? How do we create a qubit? Usually in classical computer, we used to create a bit of 0 or 1 by using the "presence "or "absence "of electricity, but we cannot do it for quantum computers, there is no "electricity a bit present or a bit absent".

We had to find a new way to describe the 0 and the 1 state.

A widely used method to build qubits is by using the **spin of an electron**, often the electron of a phosphorus atom embedded in silicon. The two possible spin orientations — **spin-up** and **spin-down** — are used to represent the states $|1\rangle$ and $|0\rangle$ respectively. But in contrast to classical bits, the electron's spin can also be in a superposition of both up and down at once, meaning it's partly $|0\rangle$, partly $|1\rangle$. This is what allows the qubit to behave fundamentally differently from a classical bit and makes quantum computing possible.

Now that we have our foundations, we can now move the interesting part I would say.

3 Literally, Shor's algorithm.

Let's first state clearly the goal of the algorithm: **Given a semiprime number$N$, find the two prime numbers $P1$ and $P2$ such that: $P1 \times P2 = N$.**

The classical way to find these numbers is just by guessing randomly, also known as the brute force method. The essence or the main idea of Shor's algorithm will be that, given a random guess g, transform this bad guess into a better guess.

3.1 Initialization.

We start by guessing randomly a number g such that $1 < g < N$. This lead to two cases:

3.1.1 We are lucky.

g is a factor of N, or shares common factors with N. In this case, we are done thanks to Euclid's algorithm.

Definition 1:

Euclid's Algorithm is a classical method for efficiently computing the **greatest common divisor (**GCD**)** of two integers. It works by repeatedly replacing the larger number with its remainder when divided by the smaller, until the remainder is zero. The last non-zero remainder is the GCD.

Note 4:

Using that, to find a factor of N, we don't need then to guess a factor, just to find a number that shares factors with N.

After finding one of the prime numbers, let's assume $P1$, compute $N/P1$ and this will give you $P2$.

3.1.2 Most probably, we won't be lucky.

And this will make us start our process of making our bad a guess a better guess. To achieve that, we will borrow a quite helpful theorem.

Theorem 0:

For any two whole numbers A and B that are coprime, there exists a power r and a multiple m such that:

$$A^r = m \cdot B + 1$$

Equivalently:

$$A^r \equiv 1 \pmod{B}$$

A good example to visualize it will be $A = 2$ and $P = 5$, we find that $r = 1$:

$$2^1 \equiv 2 \pmod 5 \qquad 2^2 \equiv 4 \pmod 5 \qquad 2^3 \equiv 3 \pmod 5 \qquad 2^4 \equiv 1 \pmod 5$$

In our case, our A is our bad guess g, and B is our semiprime number N that we want to factor, and ultimately, we know the existence of our r.

3.2 Making the bad guess a better guess.

Let's assume that we found r, such that $g^r \equiv 1 \pmod N$. Bring 1 to the left and you get:

$$g^r - 1 \equiv 0 \pmod N$$

And using our famous identity: $(a+b)(a-b) = a^2 - b^2$:

Replace a by g, and b by 1, and going from the right side of the equality to the left, we get:

$$\underbrace{(g^{r/2} - 1)}_{\text{P1}'} \cdot \underbrace{(g^{r/2} + 1)}_{\text{P2}'} \equiv 0 \pmod N$$

Notice that r has to be even, otherwise, r/2 won't be a whole number. If we found an r odd, we repeat our algorithm from the beginning.

What we found now is actually really close to what we want, instead of finding the two numbers P1 and P2 such that $P1 \times P2 = N$, we found that N divides the product of two other numbers P1′ and P2′. Which can be written as:

$$P1' \times P2' = mN \quad , m \in \mathbb{N}$$

Even more precisely, we found two numbers P1′ and P2′ that share factors with N. Thus:

$\gcd(\text{P1}', N) = \text{P1}$ or P2. (We can replace P1′ by P2′, the outcome stays the same).

And remember that thanks to our beautiful Euclide's algorithm, we are able to compute

$$\gcd(\text{P1}', N) = P1$$

easily. Compute then $\frac{N}{\text{P1}} = \text{P2}$, (and vice versa if we got that $\gcd(\text{P1}', \text{N}) = \text{P2}$). And we have successfully found our P1 and P2.

End of our algorithm.

Or maybe not?

You may feel that this is too easy, in the sense where, using this algorithm we could directly break the RSA, why are we using it? Remember also that I gave you an introduction about quantum computing, but I didn't talk at all about it at any point.

Let's recall the algorithm's main steps.

1 – Make a guess, $g < N$ that shares no factors with N.

2 – Find r such that $g^r = mN + 1$.

3 – If r is even, calculate $(g^{r/2} + 1)$ and $(g^{r/2} - 1)$. If r is odd, go back to step 1.

4 – Use Euclid's algorithm to find the greatest common divisor.

Where is the **quantum** here?

3.3 The quantum subroutine of Shor's algorithm.

Notice that we didn't discuss before how we are finding r, which is in fact that hardest (probably due to the quantum concepts) part of the whole algorithm.

Note 5:

Through this part, the quantum part of the explanation (which is still the main thing) is going to be a bit simplified, essentially when we come to some deep notions such as Quantum Fourier Transform, etc. Such quantum concepts as said before need even their own whole article. And as again mentioned before, one of the primary goals of the article is to keep the reader on board. There will be some analogies, to the classical architecture of a computer, for the sake of your understanding, please go back to section 0 and give a quick overview to the preliminaries again.

3.3.1 Initialization.

To achieve this quantum subroutine, we will need two quantum registers, as normal registers that we have in normal computers, these ones are just going to contain qubits instead of normal bits.

First register: is going to be 2n qubits, initialized as:

$$|0\rangle^{\otimes 2n}$$

The number n just determines how accurate we want our calculations to be, however, the 2nd register size depends on the first one.

Second register is going to be n qubits, initialized to: $|1\rangle$

Note that $\otimes$ in context of quantum computing doesn't mean the XOR gate as a lot of people would think. It is a tensor product. Just consider it as a normal multiplication but between the quantum states.

Note also that for the first register, we mean that all the bits are set to 0, meaning that the state of that register is: $|000\ldots000\rangle$.

But for the second register, we mean that the value of that register is one, meaning that the state of that register is: $|000\ldots001\rangle$. It is just a story of notations.

What does it do? Applying the Hadamard Transform to a $|1\rangle$ or $|0\rangle$ gives this:

Example 1. Applying this to $0\rangle$ and $1\rangle$ gives

$$H\,|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$
$$H\,|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

3.3.2 Hadamard Transform.

Let's first define how the Hadamard Transform works. The Hadamard gate is formally defined as this:

$$H = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

What does it do? Applying the Hadamard Transform to a $|1\rangle$ or $|0\rangle$ gives this:

$$H\,|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$
$$H\,|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

During this part, we will be interested essentially in its application on the 0 state, since our first register is just full of 0's.

Notice that on 0, it created exactly a state such that the probability of getting one of the states x such that x in $\{0, (2^1 - 1)\}$ (converting the decimal to binary) is exactly equal.

Let's try to apply it to 0 states with 2 qubits.

$$H^{\otimes 2}\,|00\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$$

And on 3 qubits?

$$H^{\otimes 3}\,|000\rangle = \frac{1}{\sqrt{8}}(|000\rangle + |001\rangle + |010\rangle + |011\rangle + |100\rangle + |101\rangle + |110\rangle + |111\rangle)$$

Did you notice the pattern? If not let me show it to you:

Whenever we apply the H gate to a register of size n, such that all its qubits are 0, we get a state such that the probability of measuring a state in $\{0, 1, 2, \ldots, 2^n - 1\}$ is exactly equal for all of them.

You may wonder now why we are doing that, we will see why soon.

To end this part, we will formally say that the state of our first register now is:

$$\frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle$$

3.3.3 U.

We will apply another transformation, but to the 2nd register. Let's call this transformation U. U's definition is part of the algorithm, meaning that it is not a famous transformation created from before. Formally, the definition of U is:

$$U|k\rangle = |g \cdot k \bmod N\rangle$$

With g the random guess that we picked before.

Even more formally:

$$U|k\rangle = \begin{cases} |k\rangle & \text{if } 0 \leq k < N \\ |g \cdot k \bmod N\rangle & \text{if } N \leq k < 2^n \end{cases}$$

One good property of U is that U is a unitary transformation: which means that if you take the transpose then the complex conjugate of its matrix representation, it gives you U back.

How are we going to use U? First let's look at our state now.

The quantum state of our full system (including the 1st and the 2nd register) is the tensor product $\otimes$ of both first and second register. As I said $\otimes$ is just a normal multiplication of the states. Our first register is now a sum of equal states, and our second register is still just in a 1 state. "Expanding" the expression:

$$\left(\frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle\right) \otimes |1\rangle$$

gives:

$$\frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} (|x\rangle \otimes |1\rangle)$$

Don't overthink about it, it is the exact same way as you would expand $(a+b+c)d = ad + bd + cd$. For each x (the values in each state from 0 to $2^n - 1$), we apply the transformation x times on the qubit at state 1 associated to it. It is a bit hard to understand it like that, we can use a visualization to understand it more:

For each pair $|x\rangle \otimes |1\rangle$, we apply U getting:

$$|x\rangle \otimes |1\rangle \longrightarrow |x\rangle \otimes U^x |1\rangle = |x\rangle \otimes |g^x \bmod N\rangle$$

As an example, applying U to a state $|101\rangle \otimes |1\rangle$ (101 read as $4 + 0 + 1 = 5$, recall the preliminaries) gives:

$$|101\rangle \otimes U^5 |1\rangle = |101\rangle \otimes |g^5 \bmod N\rangle$$

And again, g is our random guess that we picked before.

Notice that when applying the transformation, we will get a certain periodicity in our quantum system due to modulo properties. To visualize it even better, let us consider that our x is

going from 0 to 7 (using thus 3 qubits), that our N was 26, and our random guess was 3, after applying all what we had before, we will get:

$$\frac{1}{\sqrt{8}}(|000\rangle\,|3^0 \bmod 26\rangle + |001\rangle\,\left|3^1 \bmod 26\right\rangle + |010\rangle\,|3^2 \bmod 26\rangle +$$

$$|011\rangle\,|3^3 \bmod 26\rangle + |100\rangle\,\left|3^4 \bmod 26\right\rangle + |101\rangle\,|3^5 \bmod 26\rangle +$$

$$|110\rangle\,|3^6 \bmod 26\rangle + |111\rangle\,|3^7 \bmod 26\rangle)$$

Which exactly gives:

$$\frac{1}{\sqrt{8}}(|000\rangle\,|1 \bmod 26\rangle + |001\rangle\,|3 \bmod 26\rangle + |010\rangle\,|9 \bmod 26\rangle +$$

$$|011\rangle\,|1 \bmod 26\rangle + |100\rangle\,|3 \bmod 26\rangle + |101\rangle\,|9 \bmod 26\rangle +$$

$$|110\rangle\,|1 \bmod 26\rangle + |111\rangle\,|3 \bmod 26\rangle)$$

Note 6:

$|x\rangle \otimes |y\rangle$ is the same as $|x\rangle\,|y\rangle$

Notice also that, the period of the periodicity is 3, which is exactly the number r that we are searching for, $3^3 \equiv 1 \pmod{26}$.

Thus, if we are able to extract the periodicity from the state, we would be done.

3.3.4 The periodicity, aka r.

This is actually a way harder task than expected. When you see the representation of the state in front of you it seems obvious, however, as we said before, if we just measure now our state it is just going to give us a random state since they all have equal probability. Which is not what we want. In fact, we are not able to extract the periodicity. However, we have something called the QFT (for **Q**uantum **F**ourier **T**ransform).

The QFT is formally defined as:

$$\text{QFT}\,|x\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{\frac{2\pi i x k}{N}}\,|k\rangle$$

Where N is the number of qubits that represent x.

The QFT doesn't extract periodicity, however, it is able to extract the frequency. And remember that

$$\text{Period} = \left(\frac{1}{\text{frequency}}\right)$$

Without entering too much into the details, believe me, the QFT would need a whole article for itself, briefly: The QFT in Shor's algorithm transforms the hidden period r of $g^x \bmod N$ into measurable frequency information, allowing us to extract r from a single quantum

measurement.

Basically, the amplitudes (that represent the probability of being measured) C are **peaked** near j/r, where j is just the state that we randomly measured, since a frequency function just works with adding $1/r$ each time.

Thus, since the highest probability is near to j/r, we most probably are going to measure j/r.

But now yes we have j/r, how do we get r? If you measure $j/r = 0.312$, how can we extract r?

Luckily we already have a classical (runnable on a classical computer) algorithm that can extract r.

3.3.5 Continued Fraction Algorithm (CFA).

The best way to explain the Continued Fractions Algorithm is to give an example.

Assume that we found that j/r = 0.312

We start by writing it as a fraction:

$$0 + \frac{312}{1000}$$

We then reverse the fraction and get this:

$$= 0 + \frac{1}{\frac{1000}{312}} = 0 + \frac{1}{3 + \frac{8}{39}}$$

And we repeat the same process until we get a 1 in the numerator.

$$0.312 = 0 + \frac{1}{3 + \frac{1}{4 + \frac{1}{1 + \frac{1}{7}}}}$$

For each fraction, we get a better approximation for 0.312:

$$0.312 \approx \frac{1}{3}, \quad j = 1, \quad r = 3$$

$$0.312 \approx \frac{1}{3 + \frac{1}{4}} = \frac{4}{13}, \quad j = 4, \quad r = 13$$

$$0.312 \approx \frac{1}{3 + \frac{1}{4 + \frac{1}{1}}} = \frac{5}{16}, \quad j = 5, \quad r = 16$$

We choose an approximation of r such that it follows two conditions:

1- It should trivially be less than N

2- It should be an even number.

If none of our approximations gives a valid r, we repeat the same process from the beginning.

And this gives an end to our quantum subroutine.

And an official end of our Shor's algorithm.

4 Final thoughts: does Shor's algorithm really break RSA?

During your lecture, you went through all the Shor's algorithm, you had a whole journey with it, and I hope that I was able to make you enjoy the journey. You probably have a lot of questions in your mind now, and one of them would be that you saw with your own eyes that we have an algorithm that breaks our usual encryption. So how is our data safe?

Actually, we not really did. We are assuming that we have a quantum computer that is able to run Shor's algorithm, and we don't. Shor's algorithm would need 4100 logical qubits, or 20 000 000 physical qubits to run.

Currently, the main actor in this field is IBM, and they are aiming for 1121 physical qubits soon, far from our 20 000 000 ones. But the day we will be able to make a quantum computer with 20 000 000 physical qubits, we will definitely be able to break RSA, so the question is not if we are going to be able to, but **when**?

References

[1] P. W. Shor, "Algorithms for Quantum Computation: Discrete Logarithms and Factoring," *Proc. 35th Annual Symposium on Foundations of Computer Science*, IEEE, 1994.

[2] D. Deutsch and R. Jozsa, "Rapid Solution of Problems by Quantum Computation," *Proc. Royal Society of London A*, 1992.

[3] S. Arora and B. Barak, *Computational Complexity: A Modern Approach*, Princeton University Press, 2009.

[4] K. L. Watkins, "An Exposition on Peter Shor's Polynomial-Time Factoring Algorithm and Its Effects on Post-Quantum Cryptography."

[5] Classiq Technologies, "Quantum Cryptography — Shor's Algorithm Explained."

[6] Classiq Technologies, "Quantum Algorithms: Shor's Algorithm."

[7] Wikipedia, "Shor's Algorithm."

[8] Wikipedia, "BQP."

[9] Wikipedia, "Hadamard Gate."

[10] Wikipedia, "Quantum Fourier Transform."

[11] Wikipedia, "Euclidean Algorithm."

[12] Quantum Inspire, "Hadamard Gate."

[13] Veritasium, "How Does a Quantum Computer Work?" (video).

[14] Veritasium, "How to Make a Quantum Bit?" (video).

[15] Quantum Soar, "Quantum Computing Course: 3.8 Shor's Algorithm" (video).

Bragg's Law and X-Ray Crystallography

XXX — Author Name

Constructor University Bremen

Abstract. XXX — Abstract text goes here. Provide a concise summary of the article's contributions, methodology, and key results. This placeholder should be replaced with the actual abstract once the third paper is finalised.

1 Introduction

XXX — Introductory text goes here. Motivate the problem, provide context and background, and state the main questions addressed by this article.

2 Preliminaries

XXX — Definitions, notation, and background material needed for the rest of the article.

3 Main Results

XXX — Present the core content of the article. Include theorems, proofs, examples, and figures as appropriate.

4 Conclusion

XXX — Summarise the findings, discuss open questions, and suggest directions for future work.

References

[1] XXX — Reference 1.

[2] XXX — Reference 2.

[3] XXX — Reference 3.